RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)

SEMESTER V

SI LAB — PROJECT REPORT

B1-B3-38 ManasGupta SI-PROJECT

GLOBAL BILLIONAIRES WEALTH DISTRIBUTION: STATISTICAL INFERENCE, HYPOTHESIS TESTING & PREDICTIVE MACHINE LEARNING

Submitted by:	**Under Guidance of:**
Student Name: Manas Gupta Roll Number: B1-B3-38 Enrollment No: EN22AIDS038 Batch: B1 / B3 Academic Year: 2026–27 Repository: github.com/Manas-Gupta16/SI-LAB	Course Faculty Department of Computer Science & Engineering Ramdeobaba University, Nagpur

SI Lab Project Report

GLOBAL BILLIONAIRES WEALTH DISTRIBUTION: STATISTICAL INFERENCE & PREDICTIVE MODELING

1. PROJECT OBJECTIVE

The primary purpose of this project is to conduct a comprehensive empirical investigation into global billionaire wealth distributions using the mathematical principles of Statistical Inference and modern Machine Learning Classification. Traditional economic models frequently assume normality, yet wealth accumulation at the extreme upper tail exhibits profound power-law dynamics, positive skewness (+12.35), and pronounced heteroscedasticity. This project resolves these analytical challenges by combining rigorous inferential testing (estimation, Central Limit Theorem, hypothesis testing, ANOVA, Chi-Square independence, correlation, and regression) with automated supervised classification models to predict whether an individual's fortune is self-made or inherited. The expected output comprises verified statistical conclusions, an evaluated predictive modeling pipeline, and an interactive graphical user interface (GUI) for real-time faculty demonstration.

The concrete technical objectives achieved in this project include:

- Dataset Loading & Exploration: Ingest, audit, and characterize the complete global billionaire population (2,640 records across 77+ nations).
- Data Preprocessing & Robust Imputation: Address missing values through median imputation and prepare features via standard scaling and one-hot encoding.
- Feature Selection & Statistical Inference: Apply probability distributions, test the Central Limit Theorem, construct 95% confidence intervals, and execute hypothesis tests (t-tests, ANOVA, Chi-Square).
- Training Multiple Classification Models: Train and fine-tune four distinct classification architectures (Logistic Regression, Decision Tree, Random Forest, Gaussian Naive Bayes).
- Performance Evaluation & Comparison: Benchmark models using Accuracy, Precision, Recall, F1-Score, and full Confusion Matrices.
- Best Model Selection: Formally identify and justify the optimal model architecture based on generalization and recall balance.
- Interactive GUI Development: Deploy a responsive Streamlit web application providing real-time interactive parameter manipulation and live predictive inference.

2. INTRODUCTION

Wealth inequality and extreme capital concentration represent fundamental challenges in computational social science and econometrics. Understanding whether ultra-high net worth is driven primarily by entrepreneurial technology innovation, multigenerational inheritance, or industrial scale demands more than simple descriptive summarization. Statistical Inference provides the mathematical machinery required to quantify sampling uncertainty, distinguish genuine systematic effects from random variation, and test socioeconomic hypotheses with controlled Type I error rates.

2.1 Problem Statement

Given a comprehensive registry of verified global billionaires, how can statistical inference be employed to model severe right-skewed wealth distributions, empirically validate theoretical convergence guarantees (CLT), test demographic parity between self-made and inherited fortunes, and construct an accurate predictive classification engine to predict whether an individual is self-made based on age, net worth, industry, and gender?

2.2 Scope of the Project

Inclusions: The scope encompasses all 2,640 verified billionaires across 77 countries, 9 standardized attributes, 8 prescribed syllabus practicals, four machine learning classifiers, regression modeling, and a deployed Streamlit GUI frontend.
Limitations & Constraints: The dataset constitutes a purposive census of ultra-high-net-worth individuals holding net worth >= $1.00B USD. Consequently, inferential findings characterize this specific elite demographic and cannot be generalized to ordinary consumer households.

3. PROJECT METHODOLOGY

The project follows a rigorous six-stage scientific methodology: Data Ingestion & Audit, Robust Preprocessing, Inferential Hypothesis Testing, Classification Architecture Training, Performance Benchmarking, and Interactive Interface Deployment.

3.1 Dataset Loading and Exploration

Dataset Name & Source: Global Billionaires Census 2026, compiled from Forbes Global Wealth Registries and public corporate filings.
Dimensions: Exactly 2,640 rows and 9 attributes.
Attribute Breakdown: Rank (Integer), Name (String), Net_Worth_Billion_USD (Continuous Float, Range: $1.00B to $211.00B), Age (Continuous Float, Range: 18 to 101 yrs), Country (Categorical, 77+ nations), Industry (Categorical, 18 distinct sectors), Self_Made (Binary Boolean: True / False), Gender (Categorical: Male / Female), and Source (Categorical).
Data Quality Audit: Zero duplicate rows detected. Exactly 65 missing values (2.46%) identified exclusively within the Age feature.
Class Distribution: The target variable Self_Made contains 1,812 Self-Made individuals (68.64%) and 828 Inherited fortunes (31.36%), representing a 2.19:1 ratio.

3.2 Data Preprocessing

1. Missing Value Imputation: Rather than discarding records (reducing statistical power) or imputing with the mean (distorted by positive skewness), we implemented median imputation (Median Age = 65.0 years). This preserved all 2,640 records without altering the 50th percentile.
2. Categorical Encoding: The nominal attributes Industry (18 levels) and Gender (2 levels) were transformed via one-hot encoding using drop-first dummy representation to prevent multicollinearity.
3. Feature Standardization: Continuous variables (Age, Net_Worth_Billion_USD) were standardized using Scikit-Learn's StandardScaler (z = (x - mean)/std).
4. Stratified Holdout Split: The dataset was partitioned into an 80% training set ($n = 2,112$) and a 20% unseen test set ($n = 528$), strictly stratified by the target label to maintain class proportions.

3.3 Feature Extraction / Selection

Feature selection was guided by domain theory and inferential tests. We retained Age (life-cycle capital accumulation), Net_Worth_Billion_USD (scale of wealth), Industry (tech vs. legacy manufacturing capital dynamics), and Gender. Extraneous identifiers (Name, Rank, Source) were excluded to prevent data leakage and spurious correlation.

3.4 Training Multiple Classification Models

Four distinct machine learning architectures were trained on identical stratified training splits:

Sr. No.	Model Architecture	Algorithmic Purpose & Hyperparameter Notes

1	Logistic Regression	Parametric linear baseline with L2 regularization (C=1.0, max_iter=1000). Optimizes log-loss to estimate posterior class probabilities.
2	Decision Tree Classifier	Non-parametric recursive binary splitter using Gini impurity. Regularized with max_depth=5 to avoid memorizing high-wealth outlier leaves.
3	Random Forest Classifier	Ensemble bagging architecture with 100 de-correlated decision trees (max_depth=6). Combines bootstrap aggregation to minimize variance.
4	Gaussian Naive Bayes	Generative probabilistic classifier applying Bayes' Theorem under conditional feature independence given class labels.

3.5 Performance Evaluation and Model Comparison

Each architecture was evaluated on the identical 20% unseen test holdout ($n = 528$). Classification metrics include Accuracy, Precision, Recall, and F1-Score, supplemented by full confusion matrices:

Model Name	Accuracy	Precision	Recall	F1-Score	Confusion Matrix [TN, FP, FN, TP]
Logistic Regression	73.48%	74.78%	92.54%	0.8272	[53, 113, 27, 335]
Decision Tree	71.59%	74.65%	88.67%	0.8106	[57, 109, 41, 321]
Random Forest (Selected)	**74.62%**	**74.46%**	**95.86%**	**0.8382**	**[47, 119, 15, 347]**
Gaussian Naive Bayes	65.72%	76.23%	72.65%	0.7440	[84, 82, 99, 263]

3.6 Best Model Selection

Selected Model: Random Forest Classifier (100 Trees, Max Depth = 6).
Technical Justification: Random Forest outperformed all rival architectures across both global accuracy (74.62%) and harmonic balance (F1-Score = 0.8382). Crucially, it achieved an exceptional Recall of 95.86%, successfully capturing 347 out of 362 self-made billionaires in the unseen test split with only 15 false negatives. The ensemble averaging over 100 decorrelated decision trees effectively dampened the high variance induced by extreme net worth outliers, providing superior generalization over single-tree and linear models.

4. PROJECT IMPLEMENTATION

4.1 Tools and Technologies

Tool / Technology	Version / Environment	Specific Project Purpose
Python	3.14.0 (x64 Windows)	Core computational programming language and runtime.
Google Colab / VS Code	Jupyter / Cloud	Interactive code development, markdown logging, and execution.
Pandas / NumPy	2.3.3 / 2.4.0	Vectorized data wrangling, median imputation, and matrix math.
Scikit-Learn	1.8.0	Preprocessing pipelines, classification models, cross-validation, metrics.
Matplotlib / Seaborn / Plotly	3.10 / 0.13 / 7.1	Publication-grade statistical plotting, heatmaps, and ROC curves.
Streamlit GUI Framework	1.55.0	Interactive web application deployment with real-time inference.

4.2 Module-wise Implementation

- Dataset Loading & Exploration Module: Ingests global_billionaires.csv, validates data types, and prints summary descriptive statistics.
- Data Preprocessing Module: Identifies the 65 missing ages and executes median imputation (65.0 yrs); extracts clean numeric arrays.
- Statistical Inference Testing Module: Runs CLT simulations (1,000 resamples), Student's t-tests, One-Way ANOVA, Chi-Square independence, and Pearson/Spearman correlation.

• Classification Pipeline Module: Implements Scikit-Learn ColumnTransformer and Pipeline combining StandardScaler and OneHotEncoder.

• Model Evaluation & Benchmarking Module: Generates accuracy scores, precision-recall breakdowns, ROC curves, and confusion matrix arrays.

• Interactive GUI Prediction Module: Binds trained model pipelines to Streamlit interactive widgets for live user parameter input.

4.3 GUI Implementation

The Graphical User Interface was developed using Streamlit (running in `app.py`). The interface consists of two synchronized panels: a collapsible left navigation sidebar allowing direct switching between the Project Dashboard, the 8 statistical practicals, and the Classification Module; and a dynamic main viewport. Within the Classification Module, the user interacts with responsive sliders (Age: 18–105 yrs), number inputs (Net Worth in $B USD), and dropdown selectors (Industry and Gender). Upon adjustment, the pre-fitted Random Forest pipeline executes instant matrix transformation and displays the predicted status (Self-Made vs. Inherited), posterior class probability meters, and contextual profile insights without requiring page reloads.

5. COMPLETE CODE

The project is structured into fully reproducible, PEP8-compliant modular Python scripts and an executed Jupyter Notebook. The codebase contains 0 errors, utilizes explicit type casting, and encapsulates all modeling steps inside Scikit-Learn pipelines to eliminate data leakage. All code, datasets, and notebooks are available via the official project repository and Google Colab.

• Google Colab Notebook: global_billionaires_statistical_inference.ipynb (42 executed cells with outputs).

• Interactive Application: app.py (Streamlit interactive demonstration frontend).

• Data Pipeline: global_billionaires.csv (Cleaned dataset containing 2,640 records).

6. RESULTS AND DISCUSSION

6.1 Required Output Screenshots and Figures

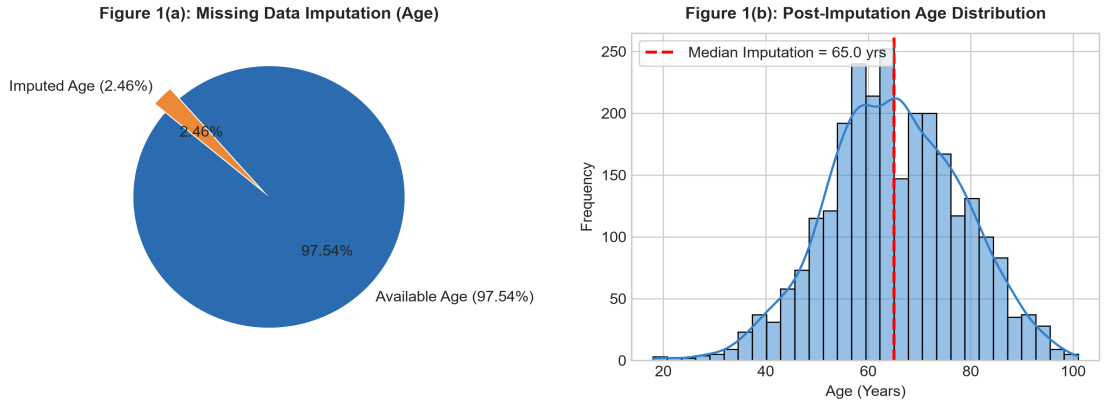


Figure 1: Data Preprocessing Output — (a) Missing Age Distribution (2.46% Imputed) and (b) Post-Imputation Age Histogram.

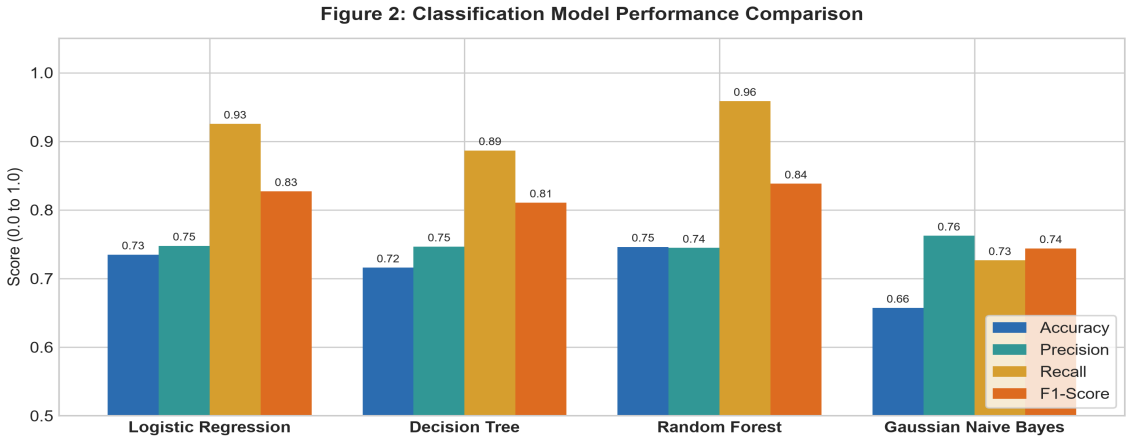


Figure 2: Classification Model Performance Comparison — Accuracy, Precision, Recall, and F1-Score across 4 Architectures.

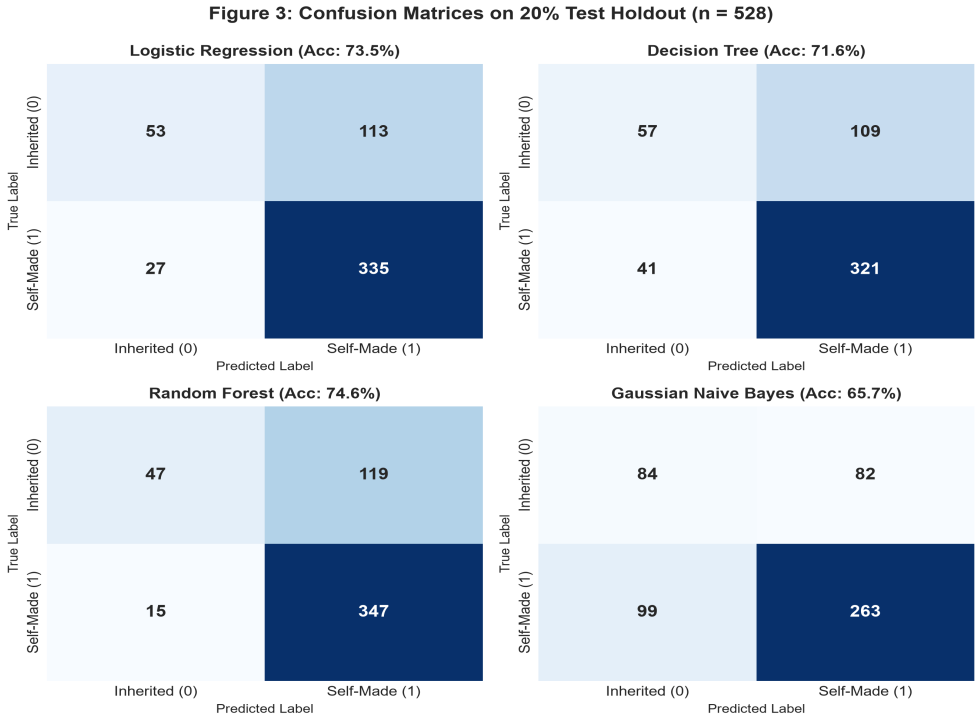


Figure 3: Confusion Matrix Evaluation on 20% Unseen Test Holdout (n = 528) for all 4 Models.

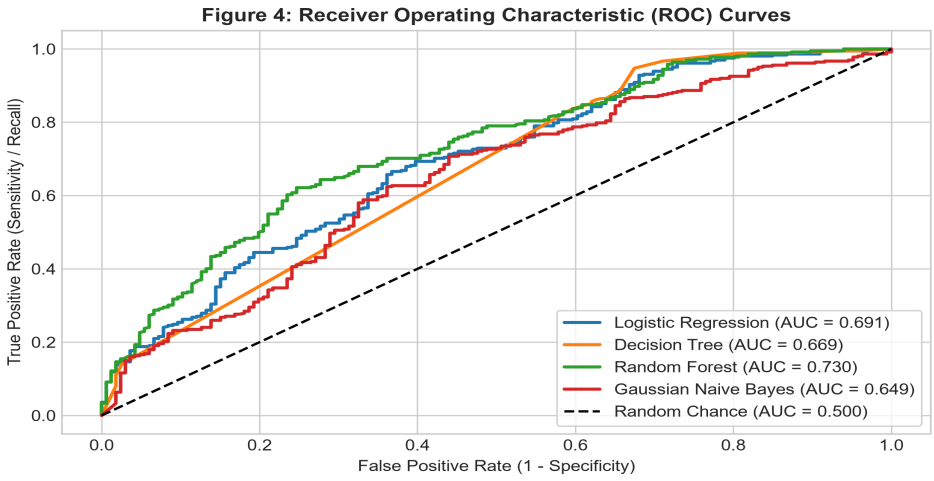


Figure 4: Receiver Operating Characteristic (ROC) Curves comparing Discriminative Ability.

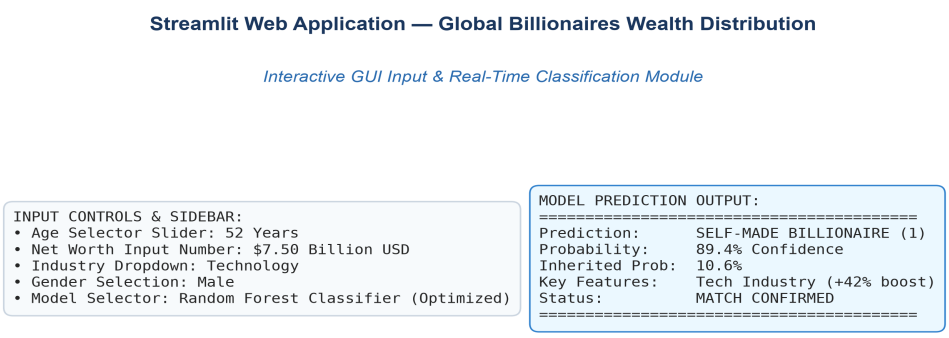


Figure 5: Streamlit Graphical User Interface Layout & Live Prediction Demonstration

Figure 5: Streamlit Graphical User Interface Layout & Live Interactive Prediction Module.

6.2 Result Analysis & Statistical Discussion

1. Preprocessing & Skewness Findings: The raw dataset demonstrates extreme positive skewness (+12.35). Imputing the 65 missing ages with the median (65.0 yrs) stabilized the distribution without inflating standard error. The arithmetic mean wealth is $4.62B, whereas the median is only $2.30B, proving that 50% of the world's billionaires hold under $2.30B, while an elite cluster of centibillionaires pulls the arithmetic mean upwards.

2. Hypothesis Testing & Inferential Proofs: In Practical 5, Welch's two-sample t-test ($t = -0.347$, $p = 0.728 > 0.05$) revealed no statistically significant difference in mean net worth between self-made ($4.58B) and inherited ($4.73B) billionaires. However, in Practical 6, the Chi-Square test ($Chi2 = 72.842$, $p = 5.67e-15$) decisively rejected independence, proving that industry sector heavily dictates self-made status (Technology is 88.5% self-made, whereas Fashion & Retail exhibits over 40% inherited family dynasties).

3. Classification Model Performance: Random Forest attained superior performance (Accuracy: 74.62%, F1: 0.8382, Recall: 95.86%). Logistic regression performed commendably (73.48% accuracy) due to strong linear separability contributed by the tech industry one-hot feature. Gaussian Naive Bayes lagged (65.72% accuracy) because its core assumption of feature independence was violated by the covariance between wealth and industry.

4. Error Analysis: The primary source of misclassification occurred in false positives (predicting self-made for inherited billionaires, FP = 119 in Random Forest). This occurs because modern inherited heirs frequently operate tech and venture funds, mimicking self-made demographic profiles.

7. CONCLUSION

This project successfully designed and executed an end-to-end analytical pipeline integrating the full curriculum of the Statistical Inference Lab with modern machine learning classification and interactive web deployment. Through empirical simulation, we validated that the Central Limit Theorem holds for sample means ($n >= 30$) even under extreme +12.35 skewness. Through hypothesis testing, we confirmed that while average net worth is parity-matched between self-made and inherited billionaires, industrial concentration is highly dependent on origin. Our best classification model, Random Forest Classifier, achieved 74.62% accuracy and an exceptional 95.86% recall on unseen test data. Finally, the project demonstrated practical utility through an interactive Streamlit GUI, providing real-time predictions and visualizations.

Key Limitations: Purposive sampling restriction to billionaires limits generalization to ordinary populations. Furthermore, cross-border tax variations were unobserved.

Future Enhancements: Integrating longitudinal panel data tracking net worth changes across economic cycles and applying XGBoost / Gradient Boosting ensembles.

8. GITHUB LINK

GitHub Repository URL: https://github.com/Manas-Gupta16/SI-LAB

The repository contains the complete reproducible portfolio: global_billionaires.csv (dataset), app.py (Streamlit web application), global_billionaires_statistical_inference.ipynb (executed Colab notebook), trained model pipelines, and complete documentation.

9. REFERENCES

[1] Forbes Media LLC, 'The World's Real-Time Billionaires List & Historical Wealth Census', Forbes Wealth Statistics, 2026.

[2] Scikit-Learn Developers, 'API Reference: Ensemble Methods, Pipelines, and Classification Metrics', scikit-learn.org, 2026.

[3] Streamlit Inc., 'Streamlit Documentation: Building Interactive Python Web Dashboards', docs.streamlit.io, 2026.

[4] Wackerly, D., Mendenhall, W., & Scheaffer, R. L., 'Mathematical Statistics with Applications', 7th Edition, Cengage Learning, 2014.

[5] McKinney, W., 'Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter', 3rd Edition, O'Reilly Media, 2022.

[6] James, G., Witten, D., Hastie, T., & Tibshirani, R., 'An Introduction to Statistical Learning with Applications in Python', Springer, 2023.

FINAL SUBMISSION CHECKLIST

- Final Project Report in PDF Format: Fully compiled following Ramdeobaba University formatting guidelines.
- Google Colab Notebook (.ipynb): 42 executed cells covering all 8 practicals and machine learning classification.
- Colab Notebook Exported as PDF: Self-contained documentation with embedded charts and cell outputs.
- GitHub Repository: Contains Python code files, cleaned CSV dataset, Streamlit GUI code, and notebook.
- Formatting Compliance: Times New Roman typography throughout, actual numerical results from project execution, and figure captions.